# PRACTICAL 9

**PHP & MySQL Database Integration, Server-Side Validation, and Secure Password Hashing**

| Student Name | \| KIRTAN KAKADIYA |
|---|---|
| **Enrollment / Roll No.** | \| 25CS026 |
| **Program / Semester** | \| 3RD |
| **Course / Subject** | \| WEB DEVELOPMENT FRAMEWORK |
| **Faculty Name** | \| BRINDA MAM |
| **Submission Date** | \| 28-10-2026 |

## 1. Aim

To implement server-side form validation, securely hash passwords, handle duplicate records, and insert user registration data into a MySQL database using PHP.

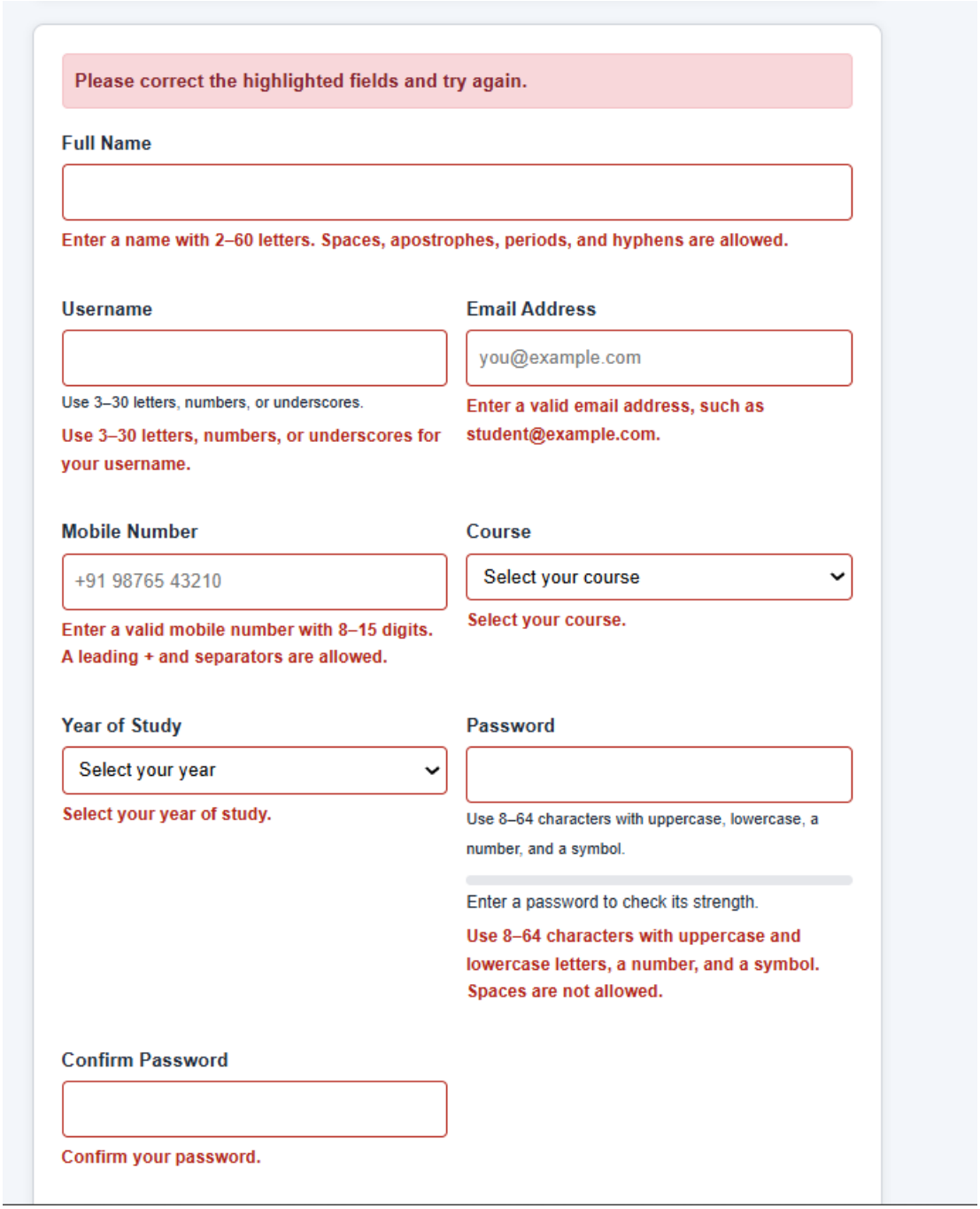
## 2. Practical Requirements

• Server-Side Validation – Validate empty fields, email formats, and password mismatches using PHP.
• Database Connection – Connect to the MySQL database (studenthub_practical8) using PDO or mysqli.
• Duplicate Handling – Query the database to prevent duplicate registrations using the same email or username.
• Secure Password Hashing – Use PHP's built-in password hashing functions (e.g., password_hash) to securely store passwords.
• Data Insertion – Insert validated and sanitized data into the student_accounts table.

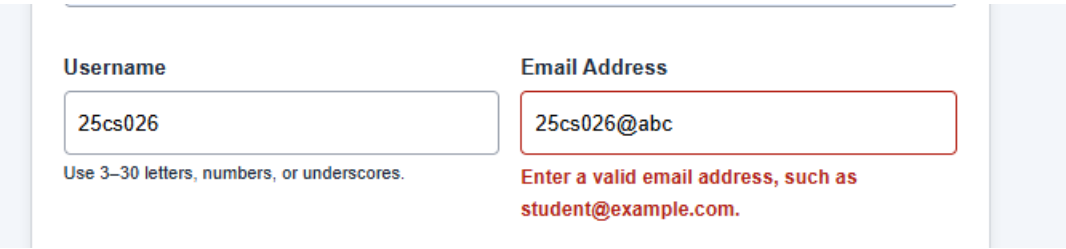
## 3. Required Screenshots / Demonstration

Insert the screenshots below in the same order as the practical instructions.

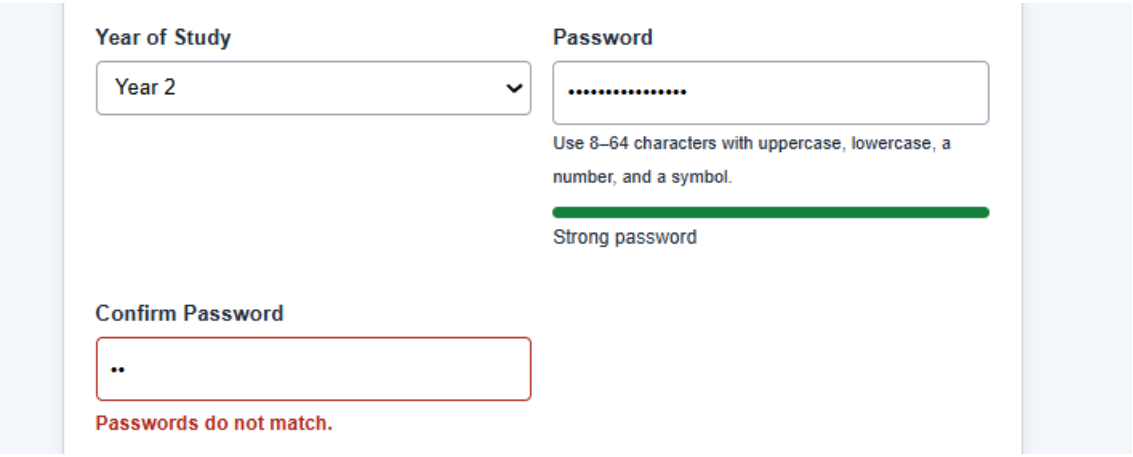
3.1 Empty-field validation



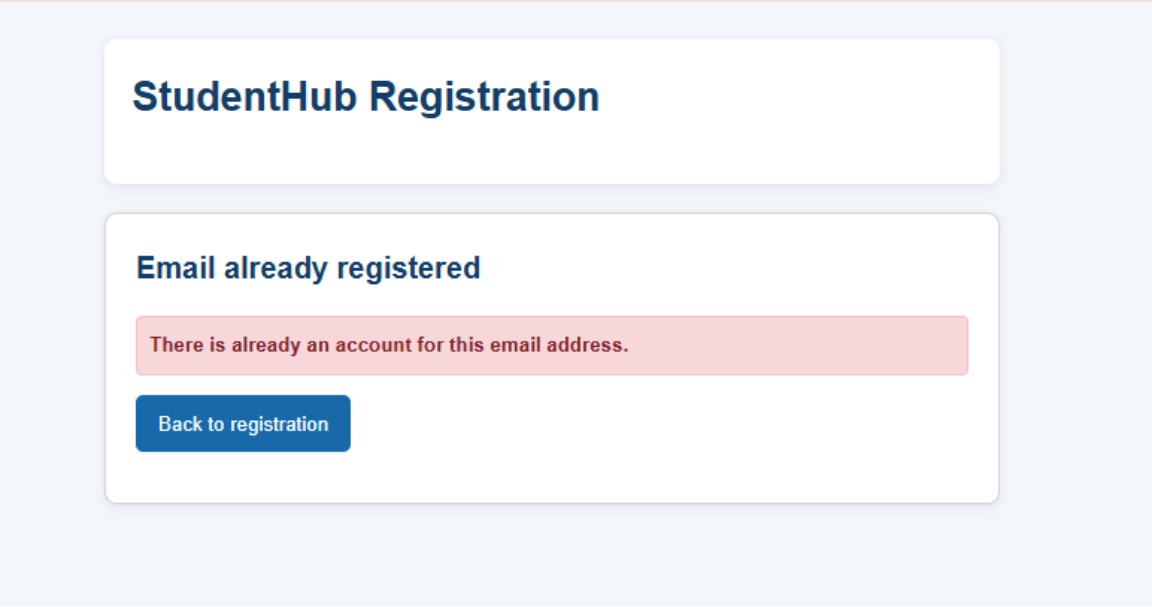

3.2 Invalid email



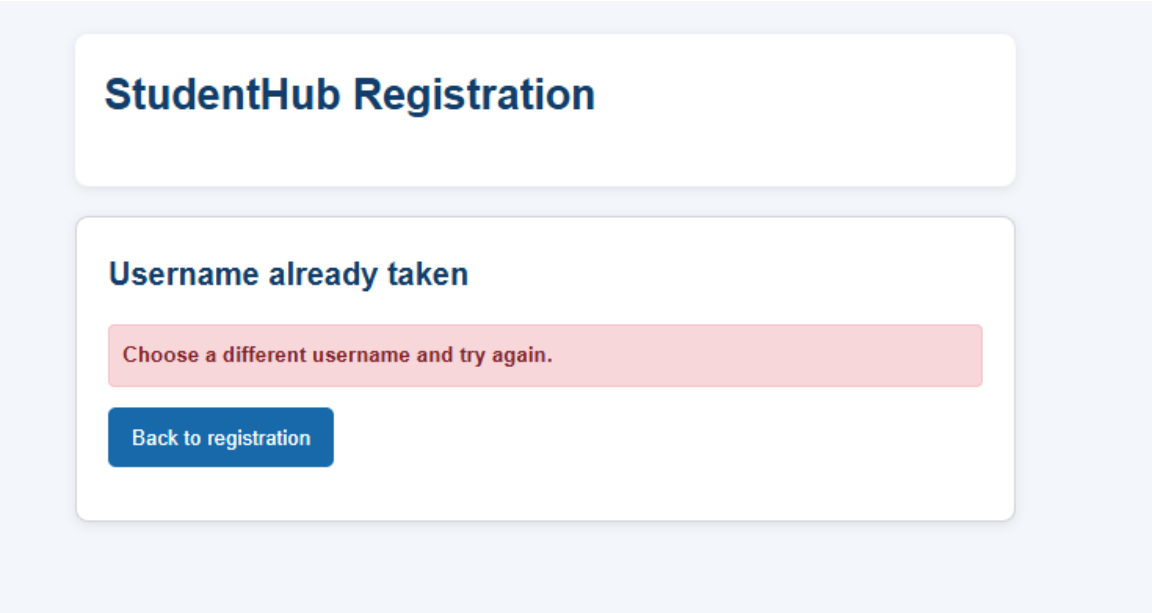

3.3 Password mismatch



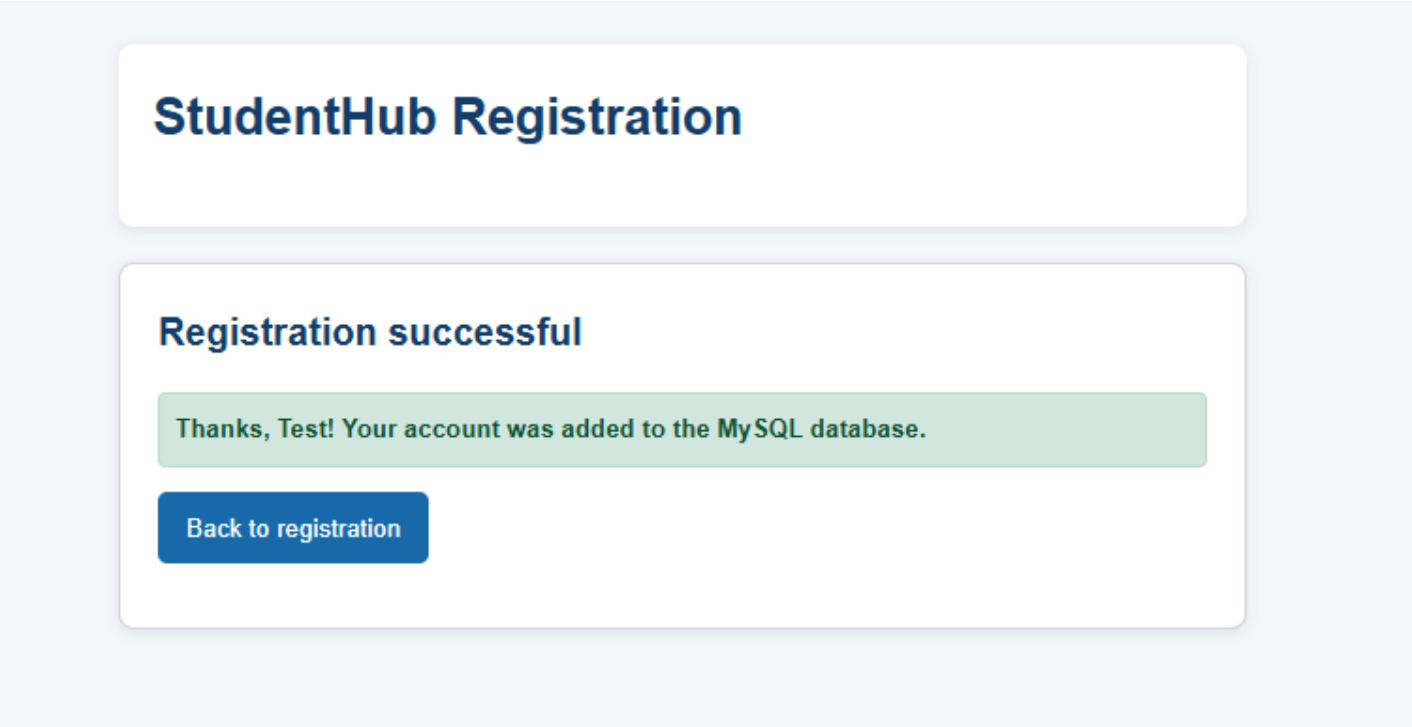

3.4 Duplicate email



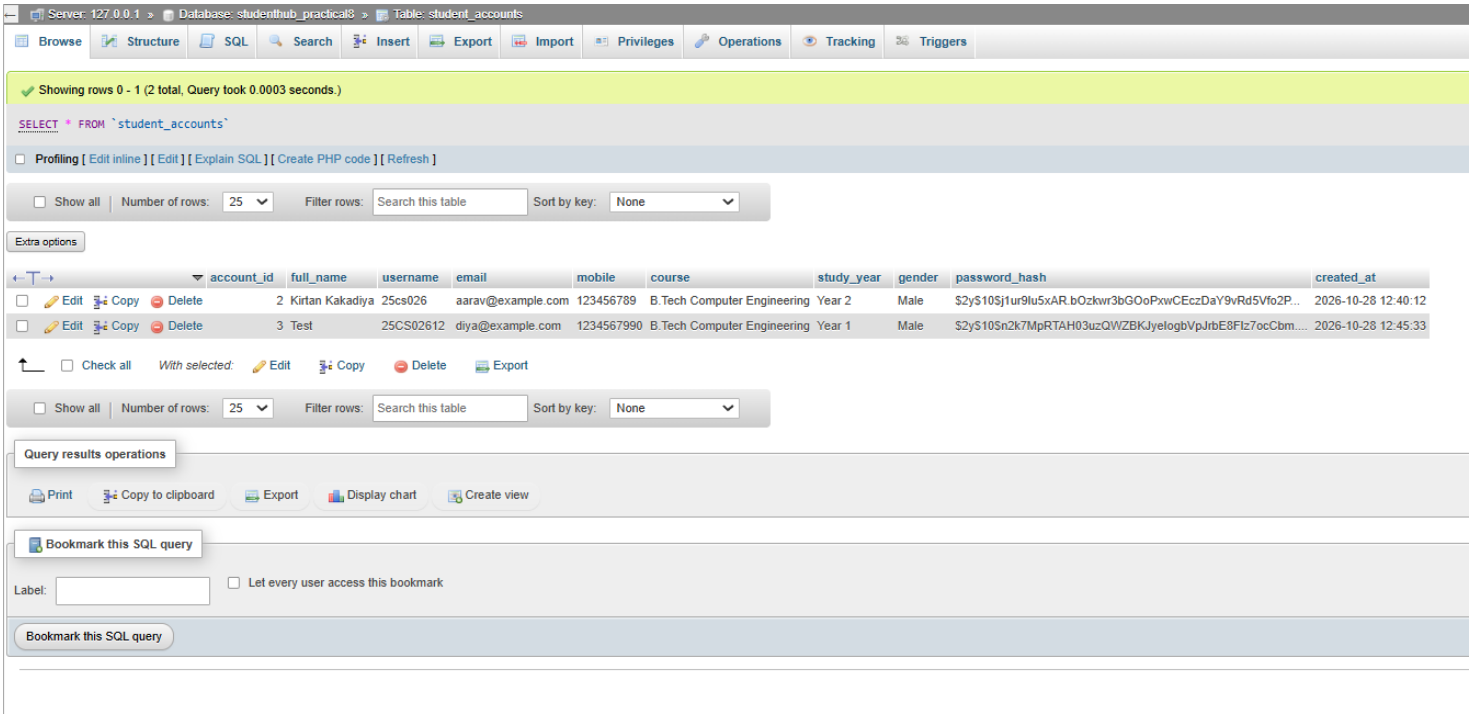

3.5 Duplicate username




3.6 Successful registration




3.7 Successful MySQL insertion and hashed password




## 4. Result

The registration form was successfully integrated with the MySQL database. Server-side validation correctly identified empty fields, format errors, and duplicate entries. Valid submissions were successfully processed, with passwords securely hashed before being inserted into the student_accounts table.

## 5. Conclusion

This practical demonstrates the critical importance of backend validation and secure database operations. By implementing server-side checks, duplicate handling, and password hashing, the application's security and data integrity are significantly enhanced, preventing common vulnerabilities such as invalid data injection and plaintext password exposure.

— End of Practical 9 Submission —